
Path of Exile

The Ultimate Gear & Crafting Guide

From Zero to Endgame

With ML-Powered Build & Gear Optimization Plan

*Sources: Path of Building codebase analysis, PoE Vault Gameplay Hub,
PoE Vault Crafting Guides, Maxroll.gg Crafting Hub, Craft of Exile,
PoE Official Forums, PoE Wiki*

April 2026 Edition

Table of Contents

- Part 1 Item Anatomy
- Part 2 Item Rarity Types and Special Categories
- Part 3 The Damage Formula
- Part 4 Universal Gear Priorities
- Part 5 Class-Specific Gear Priorities
- Part 6 The Complete Crafting Roadmap
- Part 7 Crafting Methods Deep Dive
- Part 8 Using Craft of Exile
- Part 9 Stat Priority by Build Type
- Part 10 Endgame Checklist
- Part 11 Common Mistakes
- Part 12 Essential Tools & Resources
- Part 13 ML Plan: Build Tree Optimizer
- Part 14 ML Plan: Atlas Tree Optimizer
- Part 15 ML Plan: Gear Improvement Suggester

PART 1: Item Anatomy

Source: PoE Vault Crafting Basics, Maxroll Crafting Resources, Craft of Exile Basics

The Three Layers of an Item

Layer	What It Is	Can You Change It?
Implicit	Innate to the base type	Blessed/Vaal/Eldritch
Prefix (up to 3)	+Life, +Armour, Move Speed	Yes, crafting currency
Suffix (up to 3)	Resists, Attributes, Speed	Yes, crafting currency

Hold ALT over items in-game to see prefixes vs suffixes. This is critical for crafting.

Common Prefixes vs Suffixes

Prefixes (max 3)	Suffixes (max 3)
+Maximum Life	+Elemental Resistances
+Energy Shield (flat/%)	+Attributes (STR/DEX/INT)
+Armour/Evasion (flat/%)	+Attack/Cast Speed
+Physical Damage (weapons)	+Crit Chance / Multiplier
Movement Speed (boots)	+Accuracy
Spell Damage (weapons)	+Chaos Resistance

Item Level Matters

Item level (ilvl) determines which mod tiers can roll. Higher = stronger mods.

Item Level	What It Unlocks
1-59	Low-tier mods only
60-74	Mid-tier mods, OK for early maps
75-82	High-tier, good for serious crafting
83-84	Highest tier of most mods
85-86	Special top-tier mods, specific slots

Rule: Never waste expensive currency on bases below ilvl 80.

PART 2: Item Rarity & Special Categories

Item Rarity

Rarity	Max Mods	How To Get
Normal (white)	0 explicit	Drops everywhere
Magic (blue)	1 prefix + 1 suffix	Orb of Transmutation
Rare (yellow)	3 prefix + 3 suffix	Orb of Alchemy
Unique (orange)	Fixed mods	Drops or div cards

Special Item Categories

Category	Special Property	Restrictions
Influenced	Unique mod pool	No Eldritch implicits
Fractured	Permanent mod	Eldritch OK, No Influence
Synthesised	Special implicits	No Influence, No Eldritch
Eldritch	Two extra implicits	No Influence
Corrupted	Vaal Orbed	Cannot modify further

Choose your item 'track' before crafting: you can't have Influence AND Eldritch at the same time.

PART 3: The Damage Formula

Source: Path of Building CalcOffence.lua, CalcDefence.lua

How PoB Calculates Your DPS

Total DPS = Base x (1 + Sum_Increased%) x (More1 x More2 x ...) x Speed x Crit x HitChance

The Three Modifier Types

Type	How It Stacks	Source
BASE (Flat)	All added together	Gear: +20 Phys Damage
INCREASED	All % added, applied once	Tree+Gear: 40% inc Phys
MORE	Each multiplies separately	Supports: 40% more Dmg

If you have 300% increased already, adding another 50% = only ~12.5% real DPS gain. But adding a 30% MORE multiplier = exactly 30% real DPS gain. Diversify stat sources.

Crit Effect Formula

Crit Effect = (1 - CritChance) + (CritChance x CritMultiplier)

Example: 50% crit, 400% multi = $0.5 + (0.5 \times 4.0) = 2.5x$ damage

Hit Chance (Attack Builds)

Hit Chance = Accuracy / (Accuracy + (EnemyEvasion / 5)^0.9)

85% hit chance = 15% DPS loss. Cap accuracy at 100%.

PART 4: Universal Gear Priorities

Source: PoE Vault Beginner Guide, Maxroll Atlas Progression, PoB Global.lua

Defense First (Survival = DPS uptime)

Step 1: Cap Elemental Resistances at 75%

- -30% resist penalty after Act 5, another -30% after Act 10 = -60% total
- You MUST make up this gap on gear
- Priority slots: Rings, Belt, Helmet, Boots, Gloves, Amulet

Step 2: Stack Maximum Life

- Every gear piece should have '+# to Maximum Life'
- Strength gives +2 Life per point (PoB: StrPerLife = 2)
- Targets: 3,500+ yellow maps / 4,500+ red maps / 5,500+ uber bosses

Step 3: Choose a Primary Defense

Defense	Attr	How It Works
Armour	STR	Armour/(Armour+Hit*5), diminishing vs big hits
Evasion	DEX	Entropy-based avoidance (deterministic)
Energy Shield	INT	Second HP pool, recharges after no damage
Spell Suppress	DEX	50% less spell dmg when triggered (cap 100%)
Block	Shield	Flat chance to negate hit (cap ~75%)

Step 4: Secondary Defenses

- Endurance Charges: +4% phys reduction each
- Frenzy Charges: +4% attack/cast speed each
- Power Charges: +50% crit chance each

Offense Checklist

- Weapon = most important gear piece (provides BASE damage)
- 6-link your main skill (each support = MORE multiplier)
- 100% accuracy (attacks) or cast speed (spells)
- Scale ONE damage type (don't mix unless converting)

PART 5: Class-Specific Gear Priorities

Source: PoB TreeData, CalcOffence.lua, CalcDefence.lua

MARAUDER (STR) - Juggernaut / Berserker / Chieftain

- Jugg: Stack Armour 30K+. Endurance charges = phys reduction.
- Berserker: Life leech on gear. High-risk DPS.
- Chieftain: Fire conversion. '% Phys to Fire' + fire pen.
- Priority: Weapon pDPS > Life 80+ > Resists 75% > Armour 20K+

RANGER (DEX) - Warden / Deadeye / Pathfinder

- Deadeye: Projectile damage, extra projectiles, chain.
- Pathfinder: Flask effect + flask charges. Uptime = buffs.
- Priority: Weapon crit > Spell Suppress 100% > Life+Eva > Accuracy 100%

WITCH (INT) - Occultist / Elementalist / Necromancer

- Necro: YOUR damage irrelevant. All gear buffs MINIONS.
- Elementalist: Stack elemental penetration and exposure.
- Occultist: Chaos/curse builds. ES + curse effect.
- Priority: +Gem Level > Spell Crit > Life or ES > Cast Speed

DUELIST (STR/DEX) - Slayer / Gladiator / Champion

- Slayer: Permanent overleech. Stack crit + physical DPS.
- Gladiator: Cap Block 75%/75%. Bleed explosions for clear.
- Champion: 100% Impale. Fortify = 20% less damage taken.

TEMPLAR (STR/INT) - Inquisitor / Hierophant / Guardian

- Inquisitor: Crit ignores resists. Stack crit ABOVE ALL.
- Hierophant: Mana/totem builds. MoM + mana stacking.
- Guardian: Aura stacker. Reduced Mana Reserved + aura effect.

SHADOW (DEX/INT) - Assassin / Trickster / Saboteur

- Assassin: 95%+ effective crit. Elusive for defense.
- Trickster: Hybrid ES + Evasion. Great for DoT builds.
- Saboteur: Trap/mine specialist. Trap speed = attack speed.

PART 6: The Complete Crafting Roadmap

Source: Maxroll Crafting Hub, PoE Vault Crafting Guides, Craft of Exile, PoE Forums

Phase 1: Campaign & Early Maps

Methods: Vendor recipes, Crafting Bench, Essences

- Keep resists capped - Crafting Bench for missing resist
- Get 4-link for main skill - Jeweller's + Fusing Orbs
- Upgrade weapon - Vendor: Weapon + Whetstone + Rustic Sash
- Fill life gaps - Low-tier Essences (Greed = Life)

Phase 2: Yellow Maps (T6-T10)

Methods: Essences, Bench Crafting, early Harvest

- Get a 5-link or 6-link on highest armour base
- Upgrade to ilvl 75+ bases from map drops
- Essence spam for one good mod per piece
- Bench craft the rest (resist/life)
- Quality all gear to 20%

Phase 3: Red Maps (T11-T16)

Methods: Fossils, Harvest targeted crafts, Veiled mods

- Use ilvl 83-84 bases (unlock highest mod tiers)
- Fossil craft: Pristine=life, Jagged=phys, Metallic=lightning
- Harvest 'Reforge keeping...' to lock good mods
- Unveil mods from Jun for unique bench recipes
- Lab enchant helmet for your main skill

Phase 4: Endgame Bossing & Min-Maxing

Methods: Metacrafting, Influence, Awakener's Orb, Eldritch currency

- Get Influenced bases from Shaper/Elder/Conqueror bosses
- Awakener's Orb - combine two influence mods on one item
- Eldritch implicits on non-influenced gear
- Metacraft: 'Prefixes Cannot Be Changed' + Harvest reforge
- Multi-mod: 'Can Have Multiple Crafted Mods' bench craft
- Fractured base crafting (permanent good mod)

PART 7: Crafting Methods Deep Dive

Source: Maxroll.gg Crafting Guides, Craft of Exile Basics

Essence Crafting

- Guarantees ONE specific mod, randomizes the rest
- When: Early-to-mid game; also good endgame for key mods
- Best for: Forcing life (Greed), resists, flat damage
- Tip: Use on highest ilvl base. 3 lower = 1 higher tier

Fossil Crafting (Delve)

- Biases mod pools: amplifies or blocks certain mod tags
- When: Mid-to-late game targeting specific mod combos

Fossil	Effect
Pristine	More life mods, no defense mods
Jagged	More physical mods
Metallic	More lightning mods, no physical
Frigid	More cold mods, no fire
Scorched	More fire mods, no cold
Aberrant	More chaos mods, no resistance
Dense	More ES/defence mods, no life

Harvest Crafting

- Highly deterministic: add/remove specific mod types
- When: Late game for 'finishing' items with 2-3 good mods
- Tip: NEVER start with Harvest. Use it to FINISH items.

Veiled Mods (Betrayal/Jun)

- Unique mods from unveiling (hybrid stats, special effects)
- Best for: Weapons (trigger socketed spells), jewelry
- Tip: Unveil ALL veiled items early for bench recipes

Bench Crafting

- Add one crafted mod to fill a gap (always final step)
- Best for: Capping resists, life, movement speed on boots
- Recipes from: Campaign, maps, Syndicate, Delve, Temple

Metacrafting (Advanced)

- 'Prefixes Cannot Be Changed' (2 Div) - protects prefixes
- 'Suffixes Cannot Be Changed' (2 Div) - protects suffixes
- 'Can Have Multiple Crafted Mods' (2 Div) - up to 3 crafted

Only use for expensive, near-perfect items.

PART 8: Using Craft of Exile

Source: craftofexile.com

Before spending currency in-game, ALWAYS simulate on Craft of Exile first.

Three Tools Available

Tool	What It Does	Use When
Emulator	Manual step-by-step crafting	Testing one sequence
Simulator	Thousands of sims, probability	Estimating cost
Calculator	Full mod pool with weights	Planning fossils/essences

How To Use It

1. Select your base (e.g. 'Vaal Regalia', lvl 84)
2. Set influence if applicable (Shaper, Elder, etc.)
3. Pick crafting method (Fossil, Essence, Chaos spam)
4. Define desired outcome (e.g. 'T1 Life AND T1 ES%')
5. Run simulation (1,000+ attempts)
6. Read results: Average cost, probability, sample outcomes

Pro tip: Use Calculator's 'Compute Best Selection' for optimal fossil combos.

PART 9: Stat Priority by Build Type

Source: PoB CalcOffence.lua, community meta

Physical Attack (Melee/Bow)

- 1. Weapon Physical DPS (most important)
- 2. Crit Chance + Crit Multiplier
- 3. Attack Speed
- 4. Flat Physical Damage (rings, gloves, amulet)
- 5. '% increased Physical Damage'
- 6. Accuracy -> 100%
- 7. Impale chance -> 100% (Champion)

Spell Caster

- 1. +Level to Spell Gems (each +1 = ~10% more damage)
- 2. Spell Crit Chance + Multiplier
- 3. Cast Speed
- 4. Flat Added Spell Damage
- 5. '% increased Spell/Elemental Damage'
- 6. Elemental Penetration

Damage Over Time (Poison/Bleed/Ignite)

- 1. +Gem Level (massive for DoT)
- 2. 'Damage Over Time Multiplier' (MORE multiplier!)
- 3. '% increased [Type] Damage'
- 4. Skill Effect Duration
- 5. Base hit damage (for ailment calculation)

Minion Build (Necromancer)

- 1. +Level to Minion Gems (huge stat gains per level)
- 2. 'Minions deal % increased Damage'
- 3. Minion Attack/Cast Speed
- 4. Minion Life (survival)
- 5. YOUR defenses (you still need to live)
- 6. Your personal damage = irrelevant

PART 10: Endgame Checklist

Source: PoE Vault Endgame Guide, PoB defense calculations

Stat	Yellow Maps	Red Maps	Uber Bosses
Life	3,500+	4,500+	5,500+
Fire Resist	75%	75%	75%+ (overcap)
Cold Resist	75%	75%	75%+
Lightning Resist	75%	75%	75%+
Chaos Resist	0%+	30%+	60%+
Spell Suppress	50%+	80%+	100%
Armour (STR)	10,000+	20,000+	40,000+
DPS	200K+	500K+	2M+
Move Speed	25%+	30%+	30%+
Main Skill Links	4-link	5-link	6-link

PART 11: Common Mistakes

Source: PoE Vault Beginner Guide, PoE Forums, Community wisdom

1. Too much 'increased', not enough flat damage. $200\% \text{ inc} \times 100 \text{ base} = 300$, but $100\% \text{ inc} \times 200 \text{ base} = 400$.
2. Ignoring resistance penalties. $50\% \text{ resist} = \text{DOUBLE}$ the damage taken vs 75% .
3. Bad support gems. One weak support in a 6-link is worse than a good 5-link.
4. Mixing damage types. Pick ONE and scale it unless you're converting.
5. Crafting on low-ilvl bases. Don't waste currency on $\text{ilvl} < 80$ items.
6. Skipping Craft of Exile. Always simulate expensive crafts first.
7. Ignoring gem levels. For spells/DoT, $+1$ gem level often beats any other mod.
8. Not using a loot filter. Install FilterBlade.xyz immediately.

PART 12: Essential Tools & Resources

Tool	URL	Purpose
Path of Building	pathofbuilding.community	Build planning, DPS sim
Craft of Exile	craftofexile.com	Crafting simulation
PoE Vault Guides	poe-vault.com/guides	Beginner-endgame guides
PoE Vault Crafting	poe-vault.com/guides/tag/craft..	Slot-specific crafting
Maxroll Crafting	maxroll.gg/poe/crafting	Step-by-step methods
PoE Trade	pathofexile.com/trade	Buy/sell items
FilterBlade	filterblade.xyz	Custom loot filter
PoE Wiki	poewiki.net	Mechanics reference
PoE Forums	pathofexile.com/forum	Community discussions

PART 13: ML Plan: Build Tree Optimizer

A machine learning system that maximizes utility from the passive skill tree based on your build requirements.

1. Problem Statement

Given a player's chosen class, ascendancy, main skill(s), and desired playstyle/stats, find the optimal allocation of ~120 passive skill points across the ~1,300-node passive tree to maximize a multi-objective utility function (DPS, survivability, utility).

2. Data Collection & Feature Engineering

2a. Training Data Sources

- poe.ninja build aggregator: ~50K+ builds per league with full tree, gear, and DPS data
- Path of Building exports: XML files containing exact tree allocations, computed stats, and DPS
- PoE passive tree JSON (from GGG's API): node positions, connections, stats per node
- Historical league data: track meta shifts across leagues for temporal features

2b. Feature Representation

- Tree as Graph: Each node = vertex with stat vector; edges = connections. Use adjacency matrix or edge list.
- Node features: [stat_type, stat_value, is_keystone, is_notable, is_jewel_socket, cluster_id, position_x, position_y]
- Build context vector: [class_id, ascendancy_id, main_skill_tags, damage_type, defense_type, target_content]
- Constraint features: [points_available, required_keystones, jewel_socket_targets, attribute_requirements]

2c. Label / Target

- Multi-objective score: $w1 \cdot \text{normalized_DPS} + w2 \cdot \text{normalized_EHP} + w3 \cdot \text{normalized_utility}$
- EHP = Effective Hit Pool (life * mitigation factor from CalcDefence.lua formulas)
- Utility = movement speed + flask uptime + curse effect + aura effect (contextual)
- Weights ($w1, w2, w3$) are user-configurable per request (e.g., 'boss killer' vs 'speed mapper')

3. Model Architecture Options

Option A: Graph Neural Network (Recommended)

- Architecture: Graph Attention Network (GAT) or GraphSAGE on the tree graph
- Input: Full passive tree graph + build context vector
- Output: Per-node allocation probability (sigmoid per node)
- Loss: Binary cross-entropy on node selections + constraint penalty (point budget, connectivity)
- Advantage: Naturally captures tree topology, path dependencies, and cluster relationships

Option B: Reinforcement Learning

- Environment: PoB calculation engine wrapped as gym environment
- State: Current tree allocation + remaining points + current stats
- Action: Allocate next point to an adjacent unallocated node
- Reward: Delta in multi-objective score per point allocated
- Algorithm: PPO or SAC with the tree graph as observation space
- Advantage: Handles sequential allocation naturally; can optimize path efficiency

Option C: Transformer-based Sequence Model

- Treat allocation as a sequence: [node_1, node_2, ..., node_120]
- Encoder: Build context (class, skill, gear) -> embedding
- Decoder: Autoregressive node selection with masking for valid moves
- Trained on sequences extracted from top builds

4. Training Pipeline

1. Scrape poe.ninja API for top ~50K builds per league (class, tree, gear, DPS)
2. Parse passive tree JSON from GGG API into graph structure
3. For each build, compute target score using PoB engine (batch mode)
4. Split: 80% train, 10% validation, 10% test (stratified by class)
5. Train GNN/RL/Transformer with early stopping on validation score
6. Post-training: Constraint enforcement layer (point budget, connectivity)
7. Evaluation: Compare to top-100 poe.ninja builds per archetype

5. Inference / User Interaction

- Input: User specifies class, ascendancy, main skill, and priority weights
- Optional: Lock certain nodes (keystones user wants) as hard constraints
- Output: Recommended tree allocation with per-node confidence score
- Visualization: Overlay on interactive tree (highlight allocated nodes, show stat gains)
- Iteration: User can lock/unlock nodes, adjust weights, and re-optimize

6. Integration with Path of Building

- Export: Generate PoB-compatible passive tree URL/hash string
- Import: Read existing PoB XML to use as starting point
- Validation: Run PoB engine on output tree to verify computed stats match prediction
- API: REST endpoint accepting build spec, returning optimized tree JSON

7. Key Technical Challenges

- Connectivity constraint: allocated nodes must form a connected subgraph from class start
- Cluster jewels: dynamic tree expansion changes graph topology at runtime
- Timeless jewels: transform node stats based on seed - need lookup table or sub-model
- Multi-objective balance: Pareto front exploration vs single scalar optimization
- Meta drift: model must retrain each league as tree changes and balance shifts

8. Estimated Effort

Phase	Tasks	Duration
Data pipeline	Scraper + parser + PoB batch	2-3 weeks
Model v1 (GNN)	Architecture + training loop	3-4 weeks
Constraint layer	Connectivity + budget enforce	1-2 weeks
PoB integration	Import/export + validation	1-2 weeks
UI / visualization	Interactive tree overlay	2-3 weeks
RL alternative	Gym env + PPO training	3-4 weeks

Evaluation + tuning	Benchmarks + hyperparams	2 weeks
---------------------	--------------------------	---------

PART 14: ML Plan: Atlas Tree Optimizer

A separate ML system for optimizing Atlas passive tree allocation to maximize mapping/farming rewards.

1. Problem Statement

Given a player's farming strategy (e.g., 'Expedition currency', 'Delirium XP', 'Boss rushing'), optimize the allocation of ~130 Atlas passive points across the Atlas tree to maximize returns per hour of mapping.

2. Key Differences from Build Tree

- Atlas tree is SMALLER (~130 nodes) but changes MORE frequently each league
- Optimization target is ECONOMIC (currency/hour) rather than combat stats
- No class/ascendancy dependency - same tree for all characters
- Strong interaction effects: some node combos are multiplicative
- Meta-dependent: optimal tree depends heavily on current economy prices

3. Data Collection

- Atlas tree JSON from GGG's API (node IDs, stats, connections)
- Community-shared atlas trees from poe.ninja, maxroll.gg, PoE Vault
- Economy data from poe.ninja: currency rates, divination card prices, unique prices
- Map drop data from community tracking tools (mapwatch, etc.)
- League mechanic reward tables from datamined game files

4. Feature Engineering

- Node features: [mechanic_type, reward_multiplier, spawn_chance_bonus, difficulty_modifier]
- Economy context: [chaos_rate, divine_rate, top_50_item_prices, league_day_number]
- Strategy embedding: user's selected farming goals encoded as vector
- Synergy features: pairwise node interaction scores (pre-computed from reward formulas)

5. Model Architecture

Recommended: Combinatorial Optimization + ML Hybrid

- Step 1: ML model predicts expected return per node given context (economy + strategy)
- Step 2: Constrained optimization solver (ILP/CP-SAT) selects best ~130-point allocation
- Step 3: Synergy correction: adjust predictions based on selected node combinations
- Alternative: Same GNN approach as build tree, but with economic reward signal

RL Approach

- Simulated mapping environment: run N maps, track drops, compute currency/hour
- Agent allocates atlas points to maximize simulated economic return
- Requires accurate drop simulation (can use datamined loot tables)

6. Inference

- Input: User selects farming strategy + current league economy snapshot
- Output: Optimized atlas tree + estimated currency/hour breakdown

- Rebalance: Automatically suggests tree changes when economy shifts significantly

7. Estimated Effort

Phase	Tasks	Duration
Atlas data pipeline	GGG API + economy scraper	1-2 weeks
Reward modeling	Map sim + drop tables	2-3 weeks
Optimization model	ML prediction + ILP solver	2-3 weeks
Economy integration	Live price feeds + updates	1-2 weeks
UI	Atlas tree overlay + breakdown	2 weeks
Evaluation	Compare to community meta trees	1 week

PART 15: ML Plan: Gear Improvement Suggester

An ML model that analyzes your current gear and suggests the most impactful next upgrade steps.

1. Problem Statement

Given a character's full equipment set, passive tree, and skill setup, identify which gear slot offers the highest DPS/EHP improvement per currency invested, and suggest specific mod targets for crafting or trade searches.

2. System Architecture Overview

```
User's PoB Import
|
[Gear Parser] --> Current stats per slot
|
[Slot Ranker] --> Which slot has most upgrade potential?
|
[Mod Suggester] --> What mods to target on that slot?
|
[Cost Estimator] --> How much will it cost? (trade/craft)
|
[Output] --> Prioritized upgrade list with actions
```

3. Data Collection

- PoB exports: Full gear sets with computed DPS/EHP from top builds
- Trade API: Current prices for items with specific mod combinations
- Craft of Exile data: Expected crafting cost for target mod sets
- poe.ninja economy: Item price distributions by mod tier
- Gear progression paths: Tracked across league (week 1 -> week 10 gear evolution)

4. Feature Engineering

Per-Slot Features

- [slot_type, ilvl, rarity, num_prefixes, num_suffixes, total_life, total_resist, total_dps_contribution]
- [best_possible_life_at_ilvl, best_possible_dps_at_ilvl, gap_to_best]
- [is_influenced, influence_type, has_eldritch, enchant_value]

Build Context Features

- [class, ascendancy, main_skill, damage_type, defense_type, current_total_dps, current_total_ehp]
- [gem_levels, support_gems, keystone_list, ascendancy_nodes]

Economic Context

- [slot_upgrade_market_price_p25/p50/p75, craft_cost_estimate, divine_rate, league_day]

5. Model Components

5a. Slot Impact Predictor

- Input: Current gear set + build context
- Output: Predicted DPS/EHP delta if each slot were upgraded to 'next tier'
- Architecture: Gradient-boosted trees (XGBoost/LightGBM) or small MLP
- Training: For each build, systematically swap one slot to a better version, measure delta via PoB
- This identifies: 'Your helmet is your weakest slot, upgrading it gives +180K DPS'

5b. Mod Priority Recommender

- Input: Current item in slot + build context + slot impact score
- Output: Ranked list of mod changes that maximize improvement
- Architecture: Ranking model (LambdaMART) over candidate mod improvements
- Candidate generation: Enumerate possible prefix/suffix upgrades for the slot
- Example output: '1. Add T1 flat phys (+23K DPS), 2. Upgrade life T3->T1 (+800 EHP)'

5c. Acquisition Strategy Recommender

- Input: Target mod set + current economy data
- Output: 'Buy for ~50c on trade' vs 'Craft with Essence of X for ~30c expected'
- Uses: Trade API price lookups + Craft of Exile probability estimates
- Architecture: Rule-based + ML cost predictor hybrid

6. Training Pipeline

1. Collect 100K+ builds from poe.ninja with full gear + DPS
2. For each build, generate 'upgrade scenarios' by swapping each slot
3. Compute DPS/EHP deltas using PoB engine (batch processing)
4. Train Slot Impact Predictor on (gear_set, slot) -> delta pairs
5. Generate mod-level candidates for top-impact slots
6. Train Mod Recommender on (item, mod_change) -> dps_delta ranking
7. Build Cost Estimator using trade API + Craft of Exile integration
8. Evaluate: Does the model's #1 suggestion match what top players actually upgraded?

7. User Interaction Flow

1. User imports their PoB code or pastes their account/character name
2. System parses full gear, tree, skills, and computes current stats
3. Slot Ranker shows: 'Your top 3 upgrade opportunities are: Ring1, Weapon, Boots'
4. For each slot: 'Target these mods: T1 life, T1 phys, crafted attack speed'
5. For each suggestion: 'Buy on trade for ~80c or craft with Jagged Fossil (~120c avg)'
6. Direct links to trade search with pre-filled mod filters
7. User can mark upgrades as 'done' and re-analyze for next steps

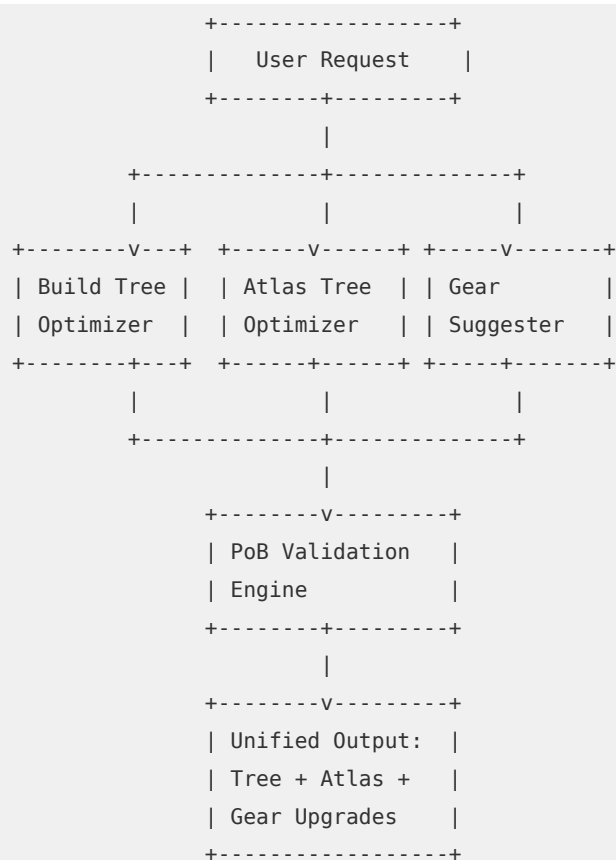
8. Advanced Features (v2+)

- Budget-constrained optimization: 'I have 5 Divine to spend, what's the best use?'
- Gear set synergy: Consider item set bonuses, stat thresholds, breakpoints
- Simulation mode: 'Show me my projected DPS after all suggested upgrades'
- League start planner: Suggest upgrade path from league start to endgame
- SSF mode: Only suggest craftable upgrades (no trade suggestions)
- Cluster jewel integration: Suggest cluster jewel notables as pseudo-gear slots

9. Estimated Effort

Phase	Tasks	Duration
Data pipeline	poe.ninja + trade API + PoB batch	2-3 weeks
Slot Impact model	Feature eng + XGBoost training	2-3 weeks
Mod Recommender	Candidate gen + ranking model	3-4 weeks
Cost Estimator	Trade + craft cost integration	2 weeks
API & UI	REST API + web frontend	3-4 weeks
PoB integration	Import/export + validation	1-2 weeks
Evaluation	User studies + accuracy metrics	2 weeks

10. Combined System Architecture



11. Technology Stack Recommendation

Component	Technology	Reason
Graph ML	PyTorch Geometric	Best GNN ecosystem
Tabular ML	XGBoost / LightGBM	Fast, interpretable
RL	Stable-Baselines3	Proven PPO/SAC impl
Optimization	Google OR-Tools CP-SAT	Constraint solver
API	FastAPI + Redis	Low-latency serving
Data	PostgreSQL + DuckDB	OLTP + analytics
Scraping	aiohttp + asyncio	High-throughput API

Frontend	React + D3.js	Interactive tree viz
PoB Engine	Lua (embedded)	Native PoB compat
Deployment	Docker + K8s	Scalable serving

This guide synthesizes data from the Path of Building calculation engine, PoE Vault's gameplay and crafting guide collections, Maxroll.gg's crafting compendium, the Craft of Exile simulator documentation, and PoE official forum community guides. The stat priorities and formulas are derived directly from PoB's CalcOffence.lua, CalcDefence.lua, and Global.lua source files. The ML plans are designed to integrate directly with the Path of Building codebase.